

Horizontal Pod Autoscaler (HPA)

Let's learn **HPA** step-by-step in a beginner-friendly KCNA notes style.

1. What is HPA?

HPA = Horizontal Pod Autoscaler

HPA automatically increases or decreases the number of Pods based on workload.

For example:

Low traffic



2 Pods

When traffic increases:

High traffic



5 Pods

When traffic decreases:

Low traffic again



2 Pods

 **Remember:**

HPA = Automatically changes the number of Pods.

2. Why is it called "Horizontal"?

Because **horizontal scaling** means adding or removing Pods.

Before:

Pod Pod



↓ Scale OUT

Pod Pod Pod Pod



You're adding **more Pods**, not making one Pod bigger.

Horizontal = $\pm$ Pods

3. Real-Life Example

Imagine a supermarket.

Normally:

 Cashier 1

 Cashier 2

Only a few customers:



Two cashiers are enough.

Suddenly:



There are too many customers.

The supermarket opens more counters:

Cashier 1

Cashier 2

Cashier 3

Cashier 4

Cashier 5

When customers leave, some counters close.

That's similar to **HPA**.

4. How HPA Works

The basic flow is:

Users / Traffic



Pods



CPU / Memory



HPA



More or fewer Pods

HPA continuously checks the configured metrics and compares them with the target.

5. Example with CPU ★

Suppose you have:

Deployment

replicas: 2

And your HPA target is:

CPU target: 50%

Currently:

Pod 1 → 80% CPU

Pod 2 → 70% CPU

The workload is higher than the target.

HPA can increase the number of Pods:

2 Pods



3 Pods



4 Pods

The exact number depends on the HPA configuration and current metrics.

6. When Traffic Decreases

Suppose later:

Pod 1 → 20%

Pod 2 → 25%

Pod 3 → 20%

Pod 4 → 25%

The workload is now low.

HPA can reduce the number of Pods:

4 Pods



3 Pods



2 Pods

This saves resources.

7. HPA Works with a Workload ★

HPA normally targets a scalable workload such as a:

- Deployment

- StatefulSet
- ReplicaSet

A common beginner example is:

HPA



Deployment



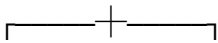
Pods

For example:

HPA



Deployment



Pod 1 Pod 2 Pod 3

HPA changes the desired number of replicas for the target workload.

8. HPA Does NOT Create New Nodes

This is **very important**.

Suppose HPA wants:

10 Pods

but your Nodes don't have enough capacity.

HPA doesn't itself create a new Node.

HPA



Needs more Pods



Cluster has no space ❌

A **Cluster Autoscaler** can potentially add Nodes:

Cluster Autoscaler



Add Node



More space for Pods

Remember:

HPA → scales Pods

Cluster Autoscaler → scales Nodes

9. HPA vs VPA ★

You should know this difference for KCNA.

HPA

Changes the **number of Pods**:

2 Pods → 5 Pods

VPA

Changes/recommends the **CPU and memory resources for Pods**:

Pod

CPU: 500m



CPU: 1000m

Easy memory:

HPA → How many Pods? 

VPA → How much CPU/RAM? 

10. What Metrics Can HPA Use?

HPA can make scaling decisions using metrics such as:

Resource metrics

- CPU
- Memory

Example:

CPU > target



Scale up

Other metrics

HPA can also work with other/custom metrics when the appropriate metrics system is configured.

For KCNA, the most important beginner example is:

CPU utilization → HPA → More/Fewer Pods

11. Important Requirement: Metrics

HPA needs metric information to make scaling decisions.

For common CPU/memory-based HPA, Kubernetes commonly uses the **Metrics API**, often provided by **Metrics Server**.

Conceptually:

Pods



Metrics



Metrics API



HPA



Scale Pods

Without the required metrics being available, HPA cannot properly make resource-based scaling decisions.

12. Simple HPA Example

Suppose your Deployment is:

my-app

with:

2 replicas

You create an HPA:

Minimum Pods = 2

Maximum Pods = 10

CPU Target = 50%

This means:

"Keep at least 2 Pods and don't go above 10 Pods. Try to maintain the configured CPU target."

Conceptually:

Low load



2 Pods

Medium load



4 Pods

High load



8 Pods

Very high load



10 Pods (maximum)

13. HPA Commands

Create an HPA

For example:

```
kubectl autoscale deployment my-app --cpu=50% --min=2 --max=10
```

Meaning:

deployment = my-app

CPU target = 50%

minimum = 2 Pods

maximum = 10 Pods

Check HPA

```
kubectl get hpa
```

You may see something like:

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
my-app	Deployment/my-app	35%/50%	2	10	2

Detailed information

kubectl describe hpa my-app

This is useful for troubleshooting and understanding why scaling is or isn't happening.

14. HPA Does Not Instantly Scale Every Time 📌

Don't think:

CPU = 51%

↓

Immediately create Pod

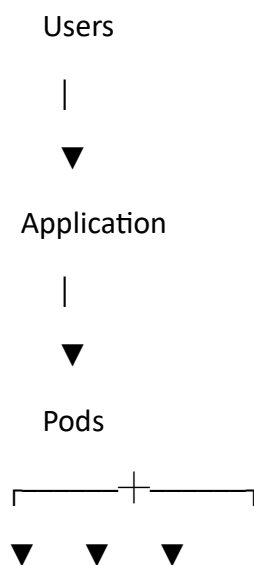
Kubernetes uses control loops and timing/stabilization behavior.

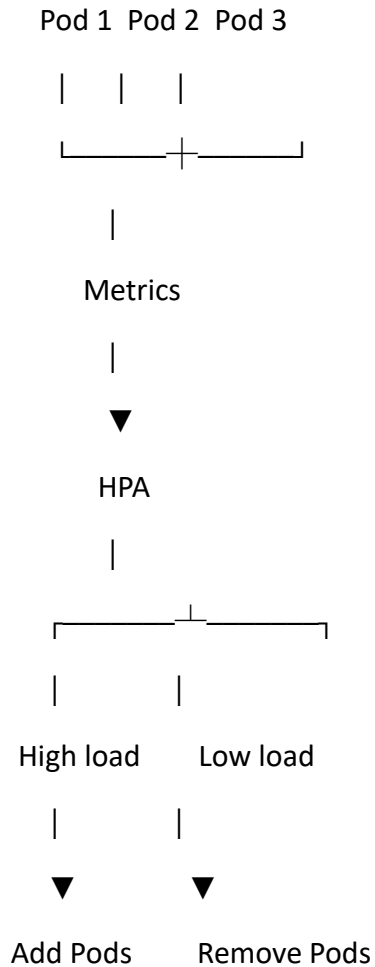
The HPA periodically evaluates the current state and adjusts the desired replica count when appropriate.

So scaling is **automatic**, but it isn't necessarily instantaneous.

15. Complete HPA Flow ★★ ★

Remember this diagram:





★ KCNA Revision Notes

HPA — Horizontal Pod Autoscaler

- **HPA = Horizontal Pod Autoscaler**
- Automatically changes the **number of Pods**.
- Used when workload/demand changes.
- Can scale **up** when demand increases.
- Can scale **down** when demand decreases.
- Commonly uses CPU/memory metrics.
- Requires appropriate metrics to be available.
- Works with scalable Kubernetes workloads such as Deployments.

- Has a **minimum** and **maximum** replica range.
- HPA does **not** create Nodes.
- Cluster Autoscaler can scale Nodes when additional cluster capacity is needed.

Super Easy Trick

HPA

|

|— H = Horizontal

| ↓

| More/Fewer Pods

|

|— P = Pod

|

|— A = Autoscaler

↓

Automatically adjusts

★ One-line exam definition

Horizontal Pod Autoscaler automatically adjusts the number of replicas of a workload based on observed metrics such as CPU or memory utilization.

Remember:

 **High demand → More Pods**

 **Low demand → Fewer Pods**

HPA = Number of Pods, not Node size.